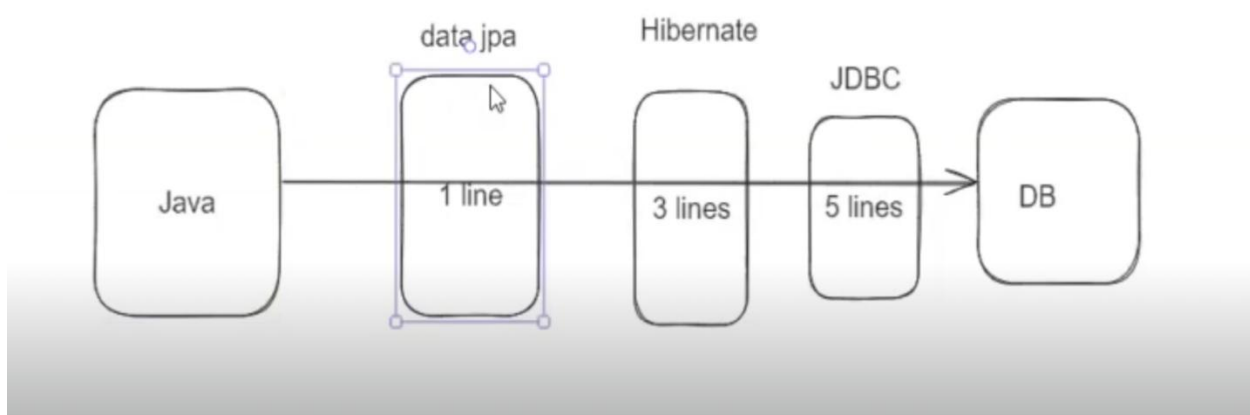


Spring Core

Core Java vs Adv Java vs Java Framework

Core Java

- ❖ Core java means J2SE
 - ❖ Core Java is base to become java developer
 - ❖ In core java Java fundamentals will be available
 - data types
 - arrays
 - strings
 - oops
 - collections
 - exception handling
 - io streams
 - lambdas
 - stream api
 - threads
-
- ❖ using core java we can develop standalone apps (one user can access a time)
Ex : notepad, calc, eclipse, intellij idea....



Adv Java (JEE)

- ❖ JEE developed based on JSE only
- ❖ JEE is called as Java Technology
 - Servlets
 - JSP
- ❖ Using this JEE we can develop web applications (multiple users can access at a time thru internet)
 - Ex: gmail, facebook, naukri, youtube
- ❖ To run web application server is required
 - Ex: Tomcat, glassfish, jboss, jetty, netty
 - 1) Web Servers (Ex : Tomcat)
 - 2) App Servers (ex: JBOSS, Web Logic, Web Sphere)

Note: App Server provides more functionalities than web server

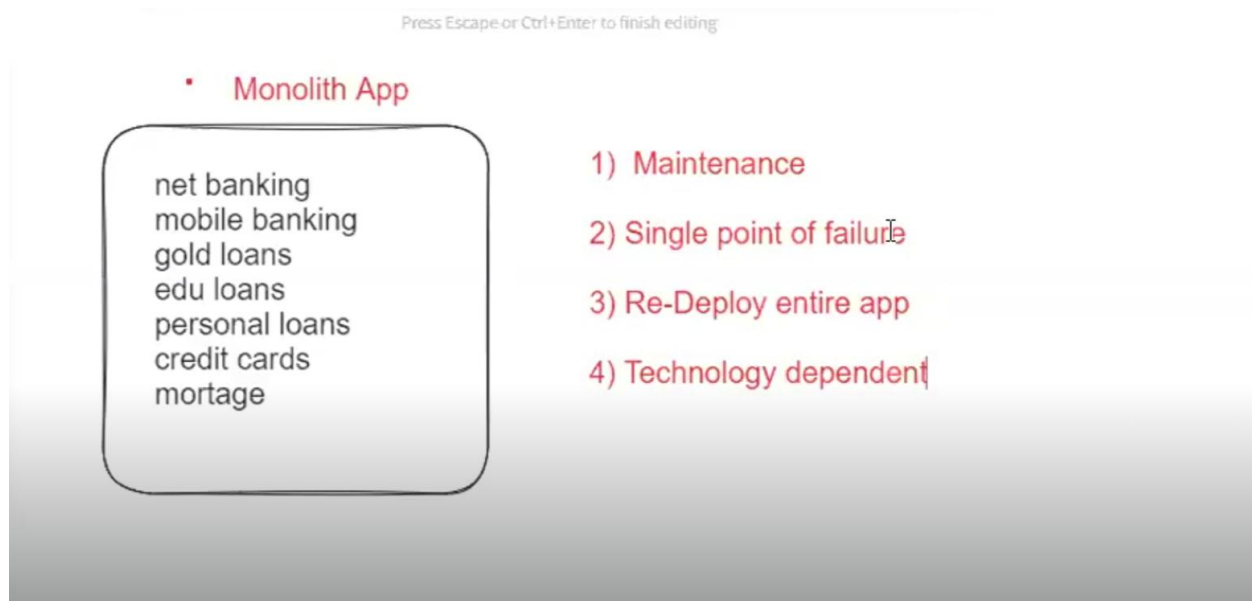
Ex : EJB container, security, conn pooling....

- ❖ Application can be developed in 2 ways
 - 1) Monolithic (outdated)
 - 2) Microservices (trending)

1) Monolithic

- ❖ Developing all the functionalities in single app
- ❖ We have below challenges in monolithic

- 1) Maintenance Difficult
- 2) Single point of failure
- 3) Re-Deploy entire app
- 4) Technology dependent



- ❖ To overcome problems of monolith, we are using microservices architecture.

2) Microservices

- ❖ It is an approach to develop application functionalities with loosely coupling.
- ❖ Application functionalities will be developed in multiple services/apis.
- ❖ Every microservice will run in separate server
- ❖ Every microservice is independently deployable and executable.
- ❖ We have below advantages with microservices

- 1) Functionality distributed
- 2) Loosely coupled
- 3) No single point of failure
- 4) Technology independent

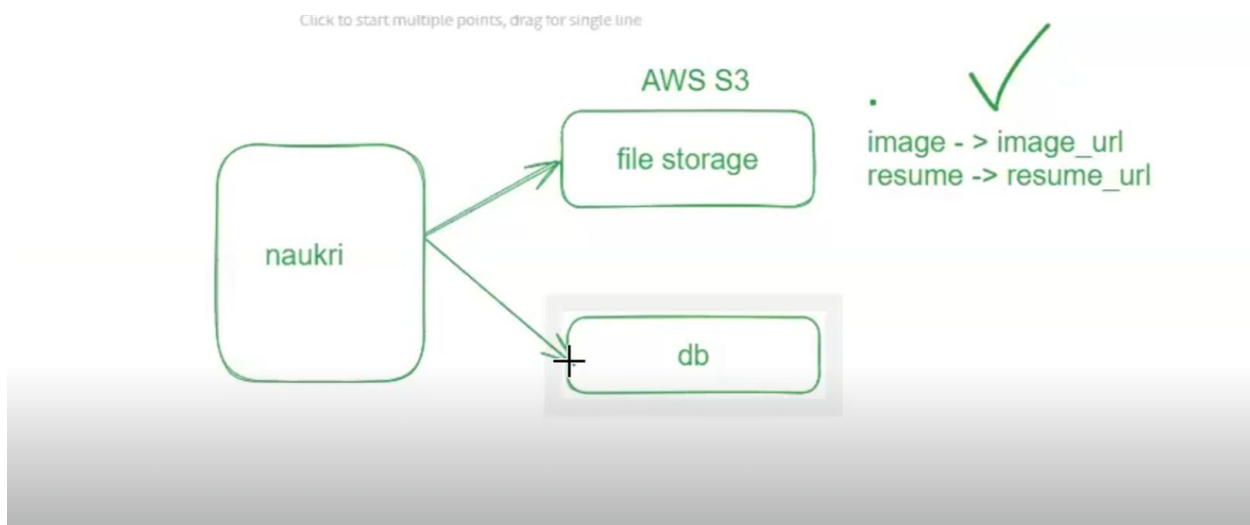
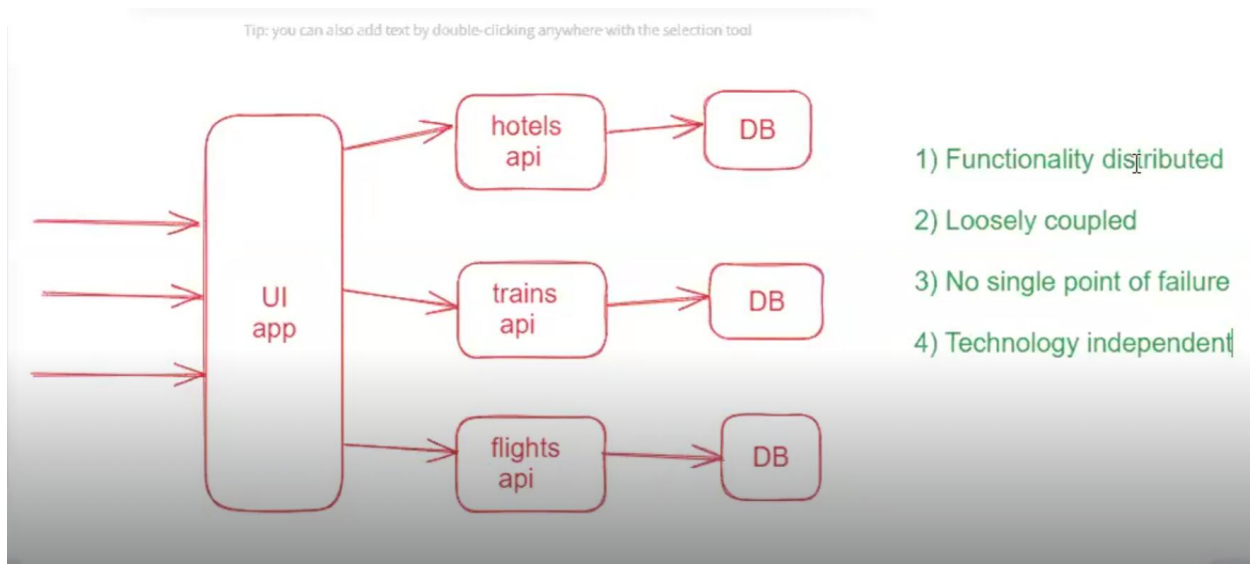
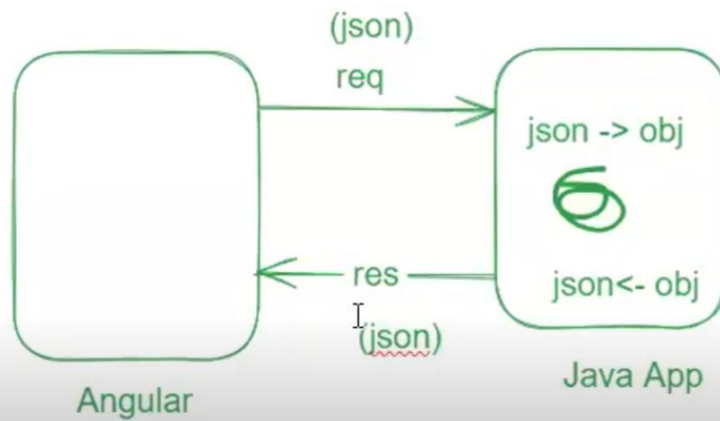


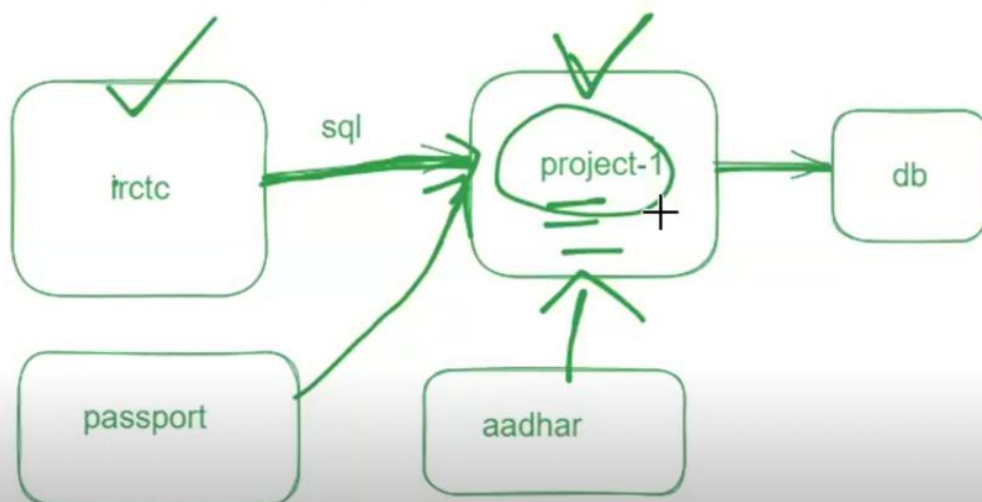
Figure 2: The flow of data between the client and the server



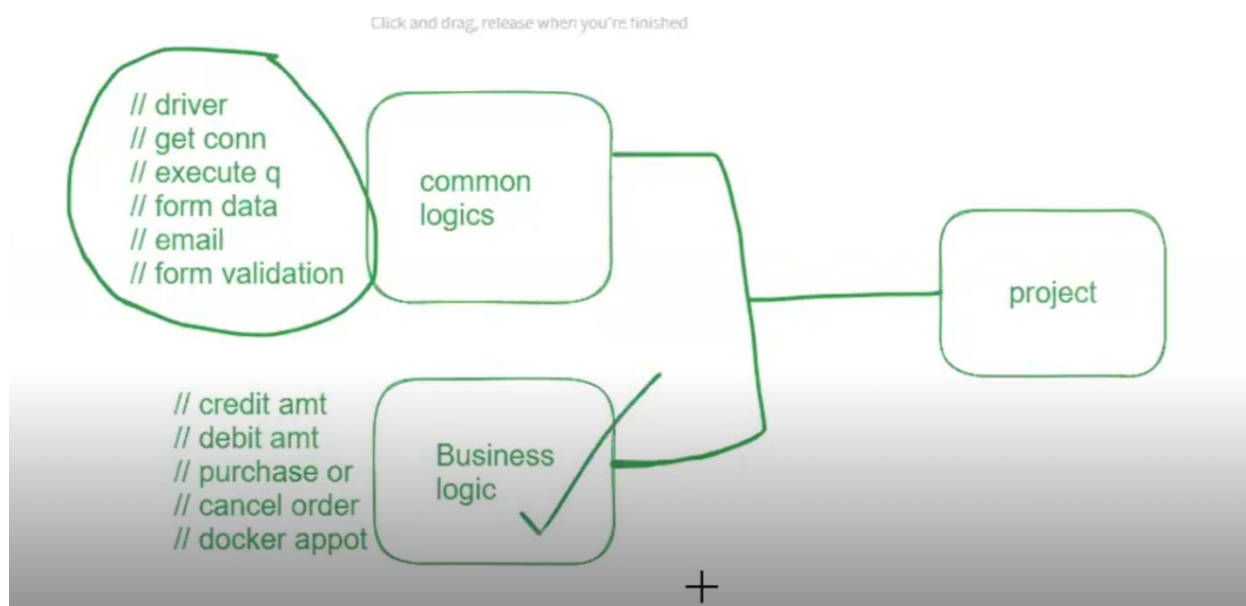
Java Frameworks

- ❖ Framework is a semi developed software

Figure 3: The flow of data between the client and the server

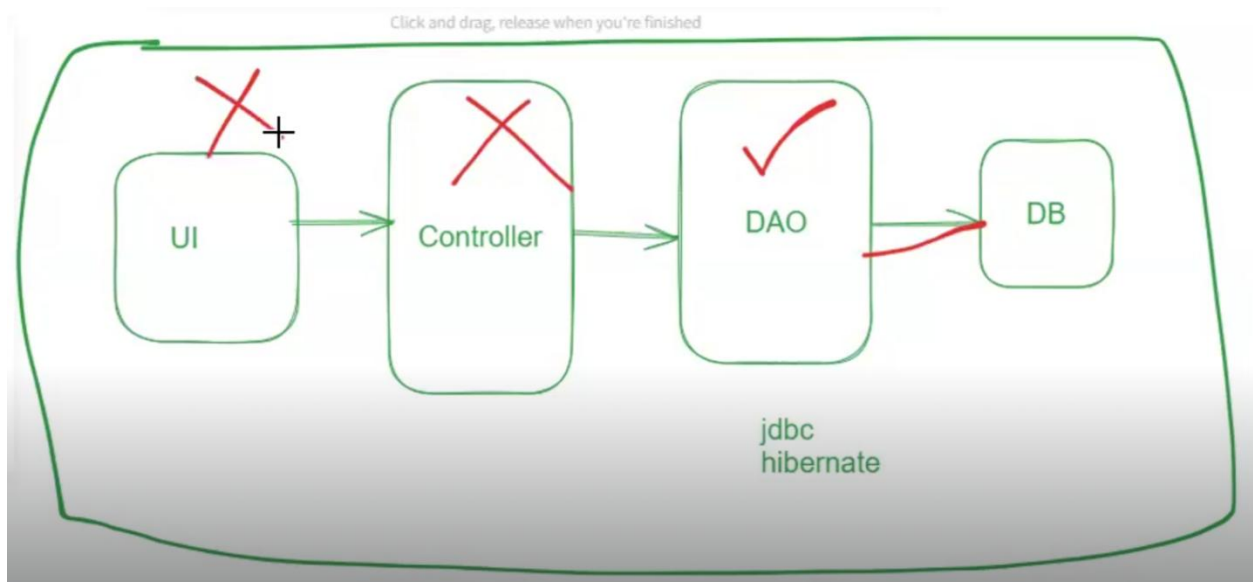
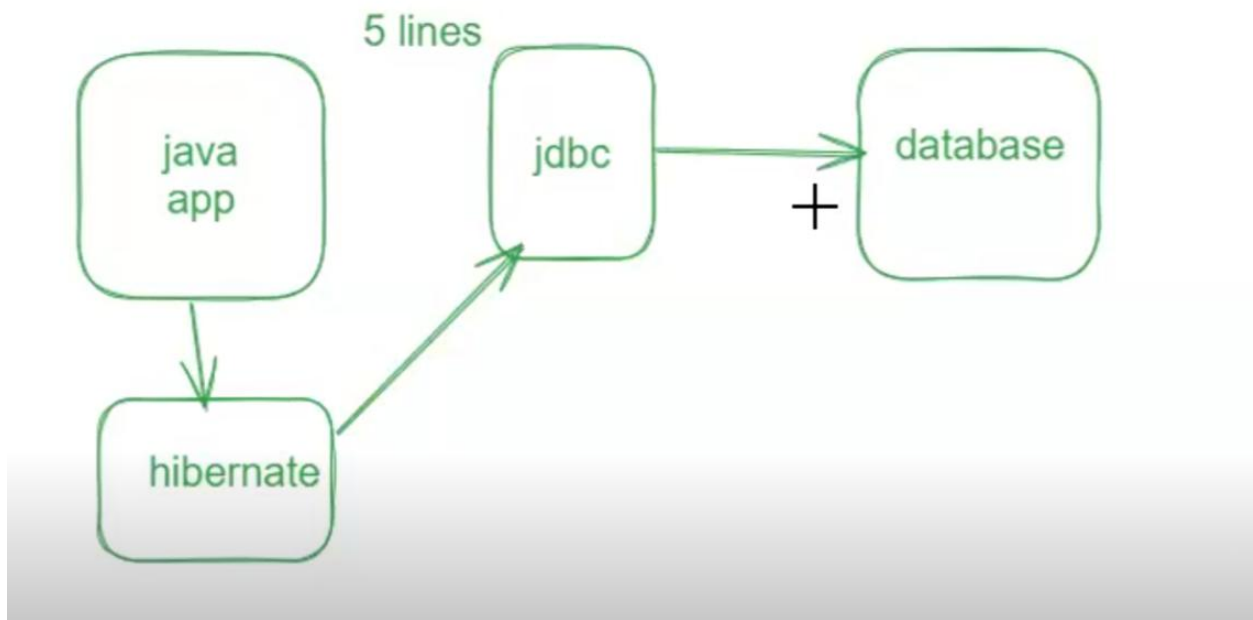


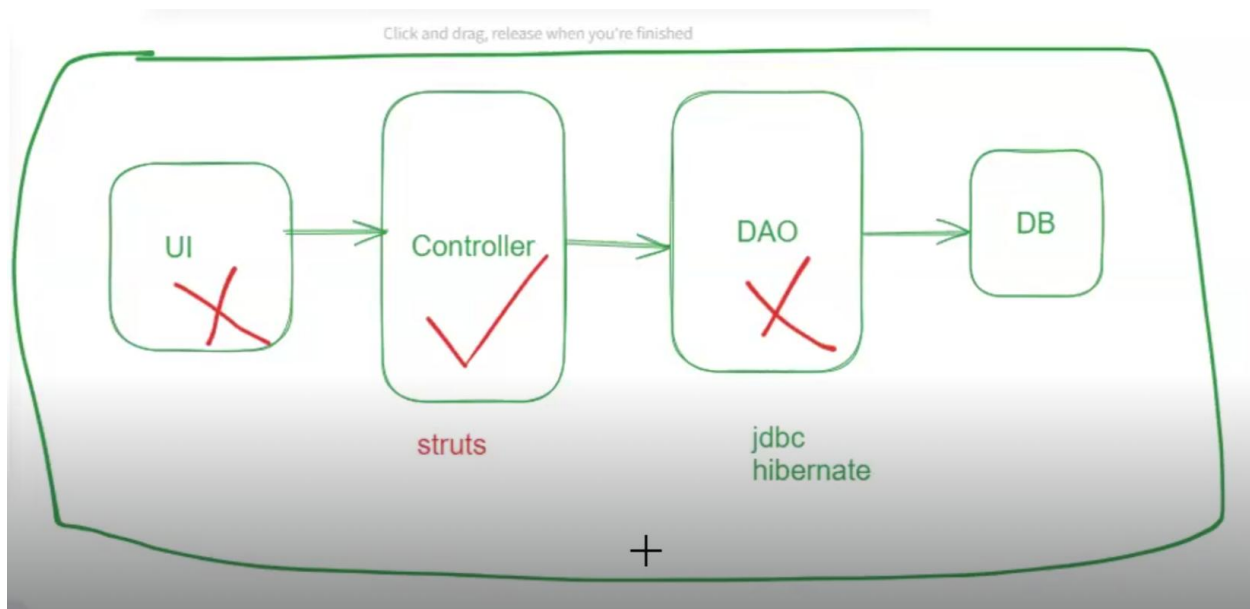
- ❖ Frameworks providing common logics for application development
- ❖ If we use frameworks then we need to focus only on business logic
- ❖ Using frameworks we can do more work in less time (productivity)



- ❖ We have several frameworks in java community
 - 1) JSF -----> outdated
 - 2) Hibernate -----> data jpa
 - 3) Struts -----> out dated
 - 4) Spring ---> SpringBoot ----> trending

Note: The above frameworks developed by using JSE and JEE.





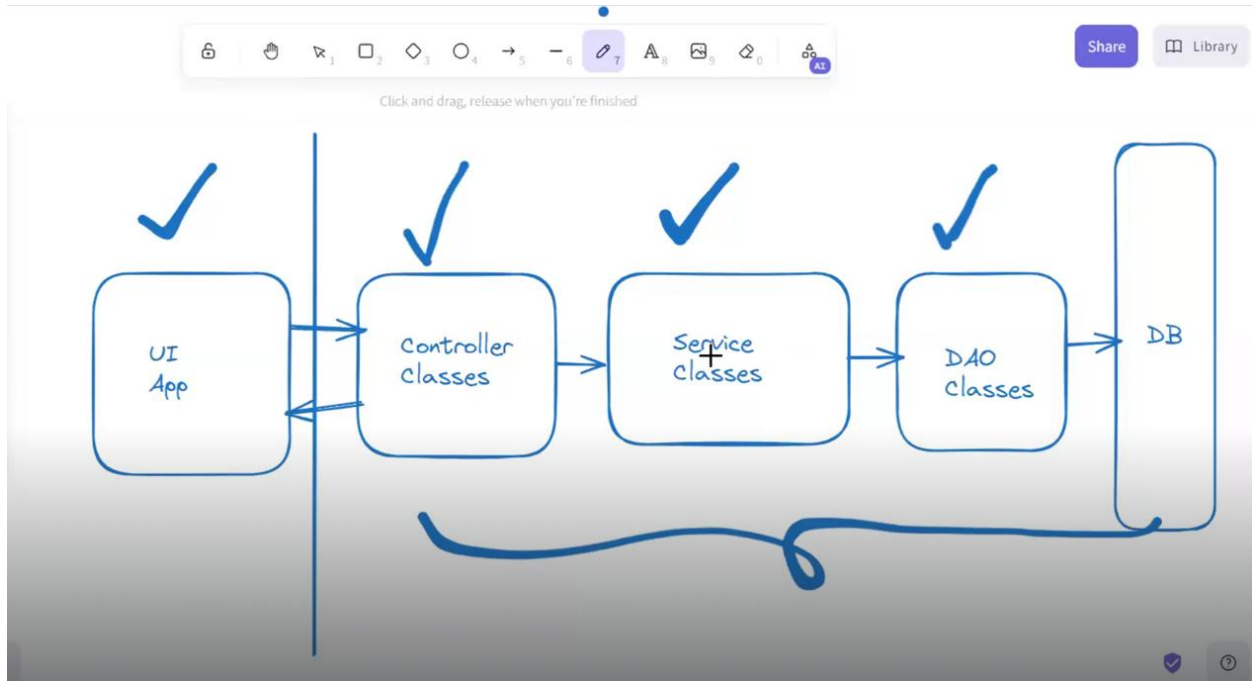
- ❖ Hibernate is an ORM framework.
- ❖ ORM means Object Relational Mapping
- ❖ Using Hibernate we can develop persistence layer
- ❖ Hibernate developed on top of JDBC.

- ❖ Struts framework developed by Apache
- ❖ Struts is used to develop only web layer

- ❖ Spring is called as application development framework
- ❖ Using spring we can develop entire application.
- ❖ Spring is free of cost & open source
- ❖ Spring framework provides common logics required for application development.

Note: Spring Boot is an extension for Spring Framework.

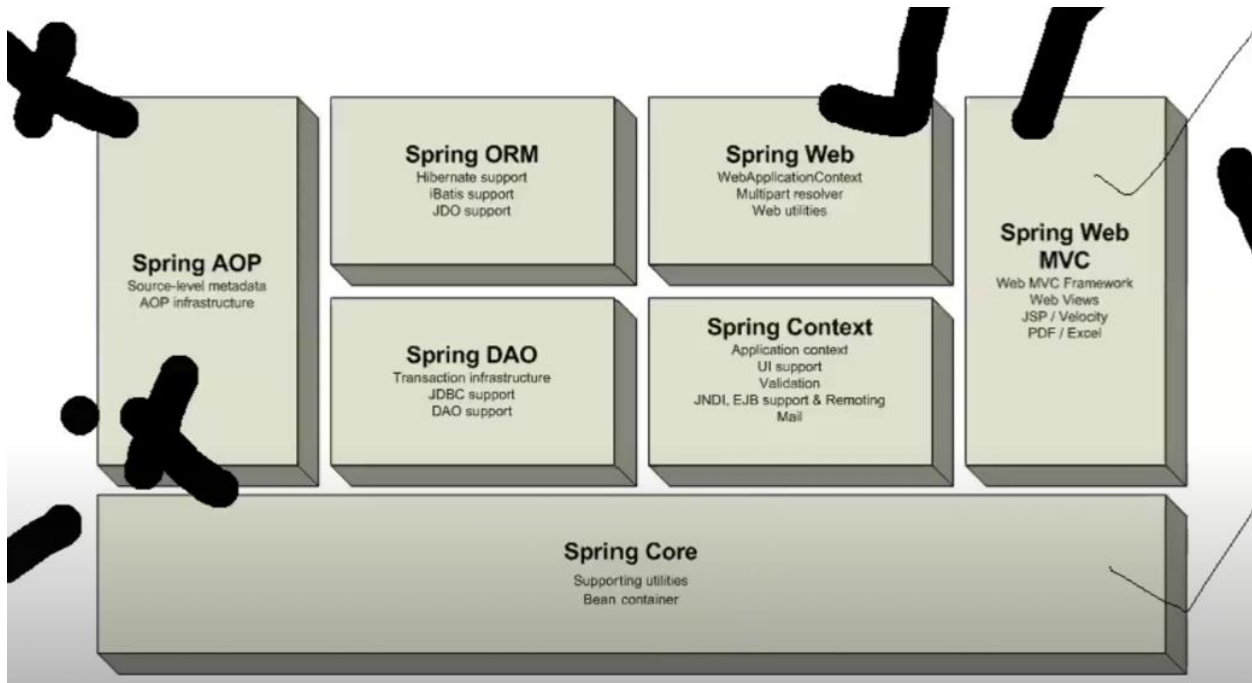
Date : 15-July-2024



What is Spring

- ❖ It is java based framework
- ❖ Spring is free of Cost & open source
- ❖ Using Spring framework we can develop several types of applications
 - 1) Standalone apps
 - 2) Web applications
 - 3) Distributed applications
- ❖ Spring is not a Single Framework
- ❖ Spring Framework developed in modular fashion

- ❖ Spring Modules are loosely coupled



Date : 16-July-2024

What is Spring Framework?

- ❖ Java Based framework
- ❖ It is free and open source
- ❖ Loosely coupled
- ❖ It is developed modular fashion

- ❖ Spring first version released in 2004 (1.x)
- ❖ The current version of spring is 6.x
- ❖ Spring framework under license of VMWare
- ❖ Below Modules in spring framework

- 1) Spring Core
- 2) Spring Context
- 3) Spring DAO
- 4) Spring ORM
- 5) Spring AOP
- 6) Spring Web MVC
- 7) Spring REST
- 8) Spring Security
- 9) Spring Cloud
- 10) Spring Batch
- 11) Spring Scheduling...

Note : Spring framework will not force us to use all the modules to develop our application.

=> We can pickup any spring module that is required for our project and we can use it.

1) Spring Core

- ❖ Base module of spring framework
- ❖ Providing fundamental concept of spring framework
 - 1) IOC Container
 - 2) Dependency Injection

3) Auto wiring

2) Spring Context

- ❖ Provide support for configurations required for application

3) Spring DAO

- ❖ DAO stands for Data Access Object
- ❖ It is used to communicate with database
- ❖ Spring DAO module providing predefined classes to simplify dao layer development.

java app ----> spring dao ----> jdbc ---> database

Ex : jdbcTemplate.execute(sql);

4) Spring ORM

- ❖ ORM stands for Object Relational Mapping
- ❖ ORM is used to perform DB operations using objects.

hibernate.save(empObj);

java app ---> data jpa ---> orm -> hibernate --> jdbc -> db

Note: Spring DAO & Spring ORM modules are used for database connectivity only

- ❖ Spring DAO => represents data in text format
- ❖ Spring ORM => represents data in objects format

5) Spring Web MVC

- ❖ MVC means Model , View & Controller
- ❖ This module is used to develop web applications
 - ◆ Customer ----> application (C2B)
- ❖ It supports for JSP and Thymeleaf as presentation technologies.

Note: JSP and Thymeleaf are outdated.

6) Spring REST

- ❖ It is used to develop REST APIs (webservices)
- ❖ REST API means distributed application.
- ❖ If one application is communicating with another application then it is called as Distributed application.

Ex: passport ----> aadhar

gpay ---> bank app

7) Spring Security

- ❖ It is used to implement security in our application
- ❖ It provides below functionalities
 - 1) Authentication (who can login ?)
 - 2) Authorization (what they can access ?)

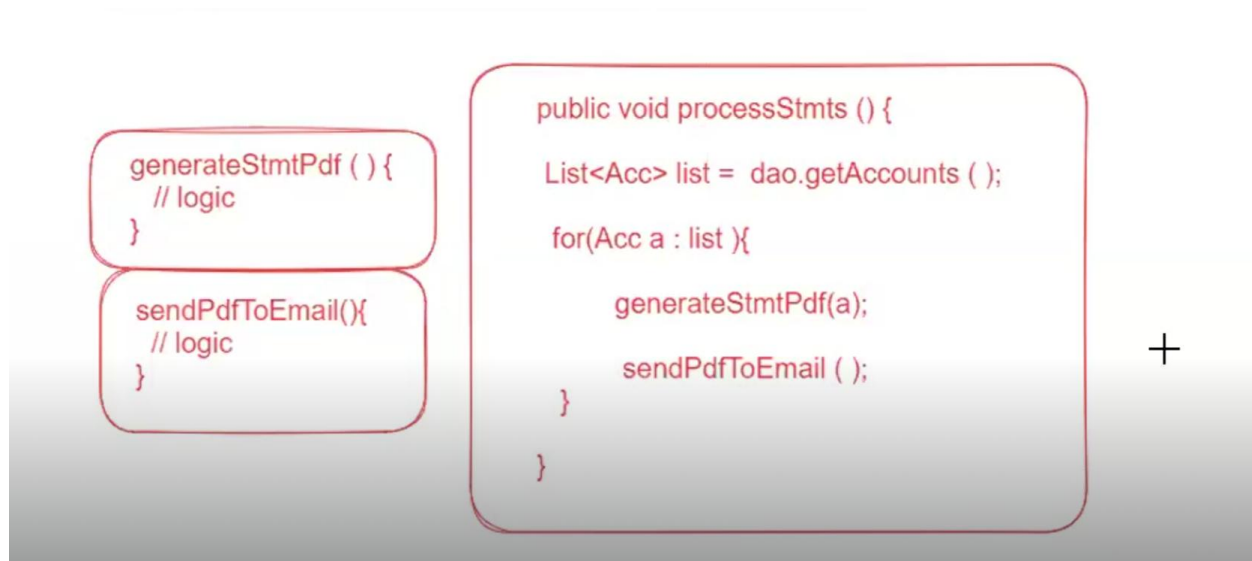
8) Spring Cloud

- ❖ It provides resources to develop microservices based applications

Ex: Discovery server, api gateway, circuit breaker, config server....

9) Spring Batch

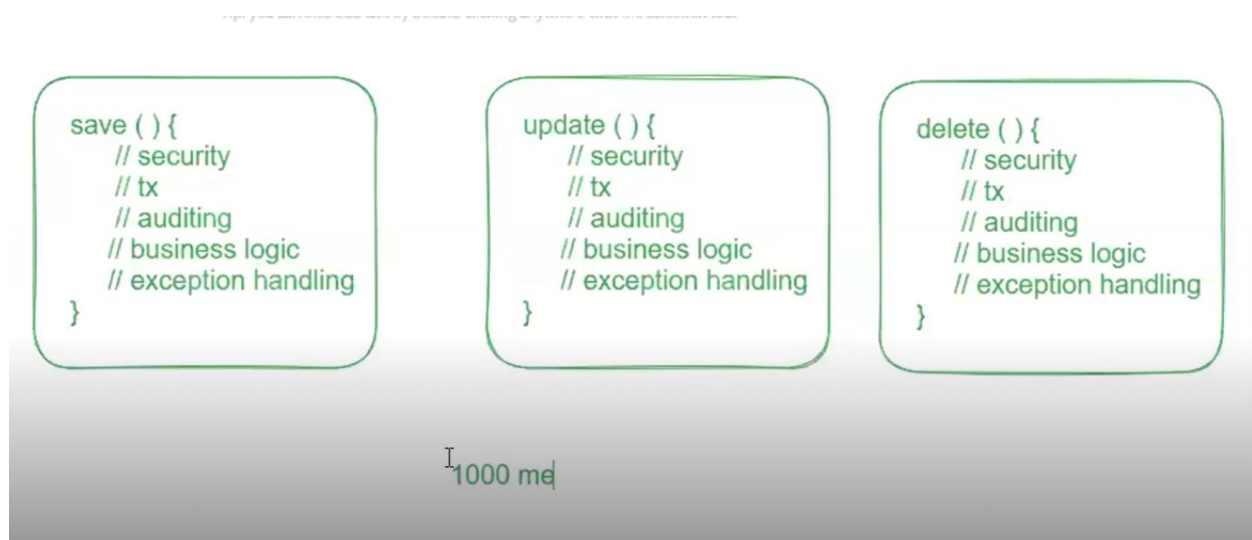
- ❖ It is used to implement batch processing (bulk operation)
- ❖ usecases:
 - 1) Bank account stmt generation
 - 2) Credit card bill generation
 - 3) postpaid mobile bill generation

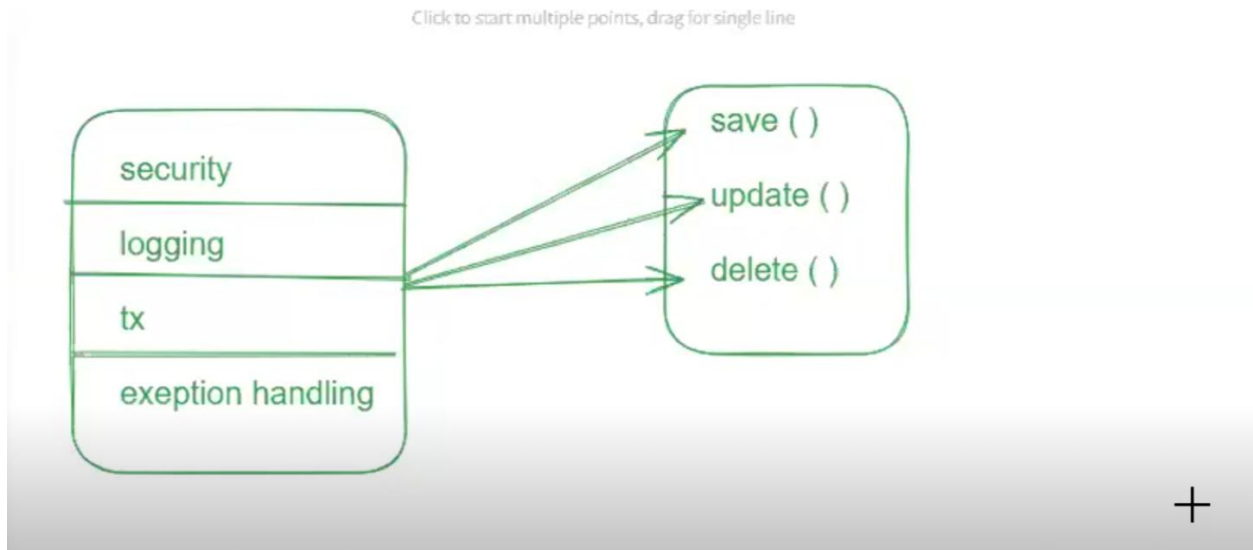


10) Spring AOP

- ❖ AOP stands for Aspect oriented programming
- ❖ It is used to separate cross cutting logics from business logic.

cross cutting logics : security, tx, auditing, logging ...





11) Spring Scheduler

- ❖ It is used to schedule task execution.

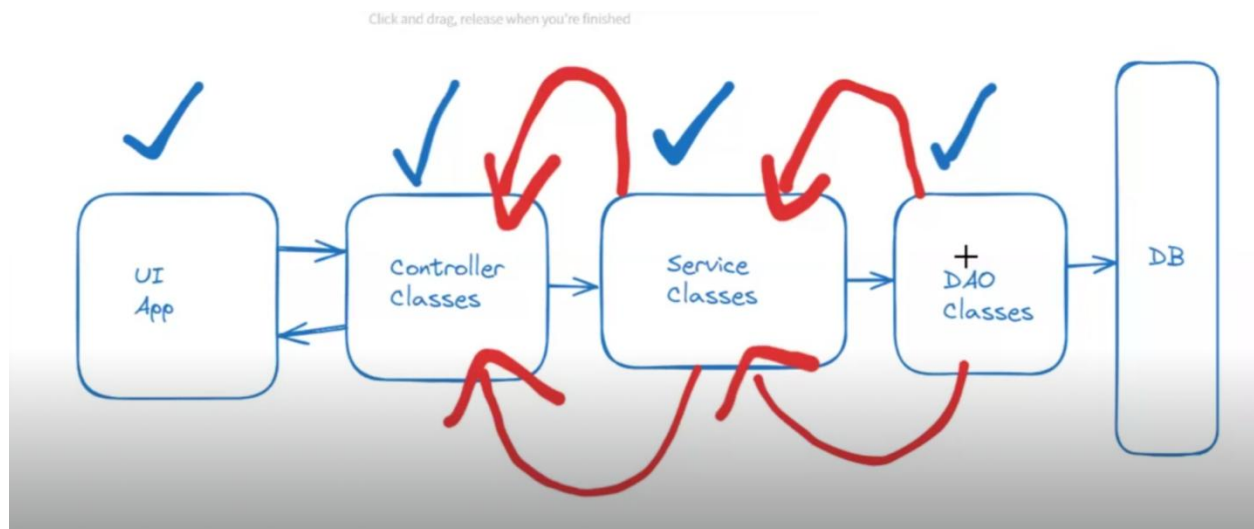
Spring Core Module

Date : 17-July-2024

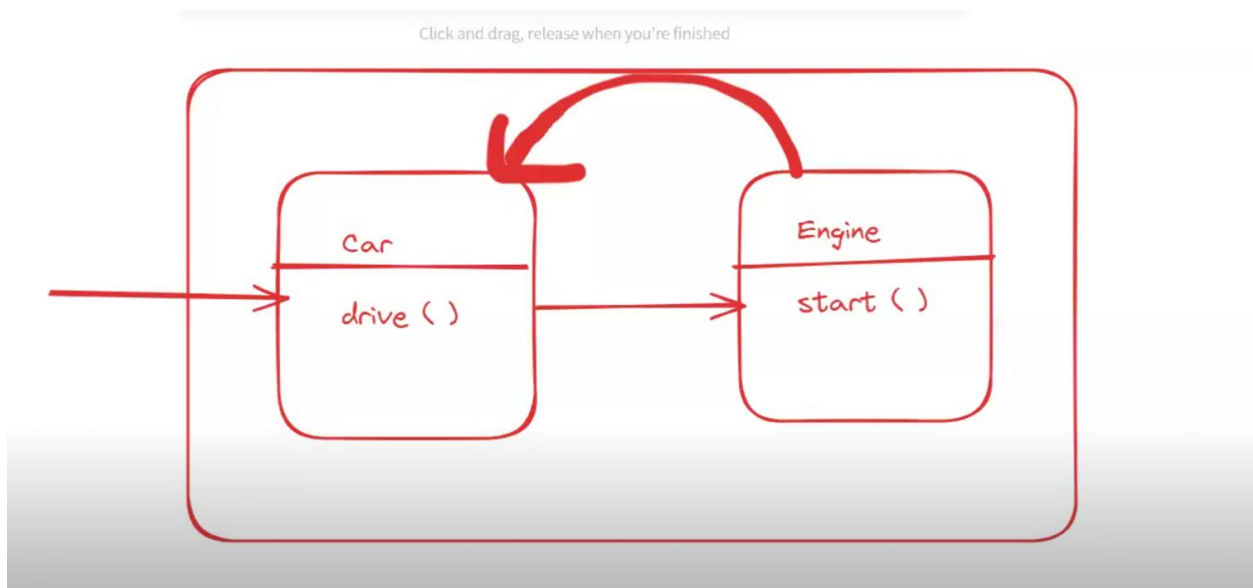
Spring Core module

- ❖ Base module of Spring Framework
- ❖ Providing fundamentals of spring framework
 - 1) IOC
 - 2) DI
 - 3) Auto wiring

- ❖ Spring Core module is used to manage our classes in the project.
- ❖ In a project we will have several classes
 - 1) controller classes (req & resp)
 - 2) service classes (business logic)
 - 3) Dao classes (db communication)
- ❖ in project execution process, One java class should call another java class method
- ❖ In 2 ways one java class can call another java class method
 - 1) Inheritance (IS-A)
 - 2) Composition (HAS-A(Object Creation))



IS-A Relation



- ❖ Extend the properties from one class to another class
- ❖ sub class can access super class method directly

Engine - super class

Car - sub class

Note: Car class can access engine class method

```
class Engine {  
    public int start() {  
        // logic  
        System.out.println("Engine started..");  
        return 1;  
    }  
}
```

```

}

class Car extends Engine {
    public void drive() {
        int engstatus=super.start();
        if(engstatus==1)
            System.out.println("Journey Started..");
        else
            System.out.println("Engine Problem...");
    }
}

public class Test {
    public static void main(String[] args) {
        Car c=new Car();
        c.drive();
    }
}

```

- ❖ In the above approach car is extending properties from Engine class.
- ❖ In future car can't extend props from other classes bcz java doesn't support multiple inheritance.
- ❖ To overcome problem of IS-A relation we can use HAS-A relation.

HAS-A relation

- ❖ Create object and call the method
- ❖ Inside car class create object for Engine class and call eng class method.

```

class Engine {
    public int start() {
        // logic
        System.out.println("Engine started..");
        return 1;
    }
}

class Car {
    public void drive() {

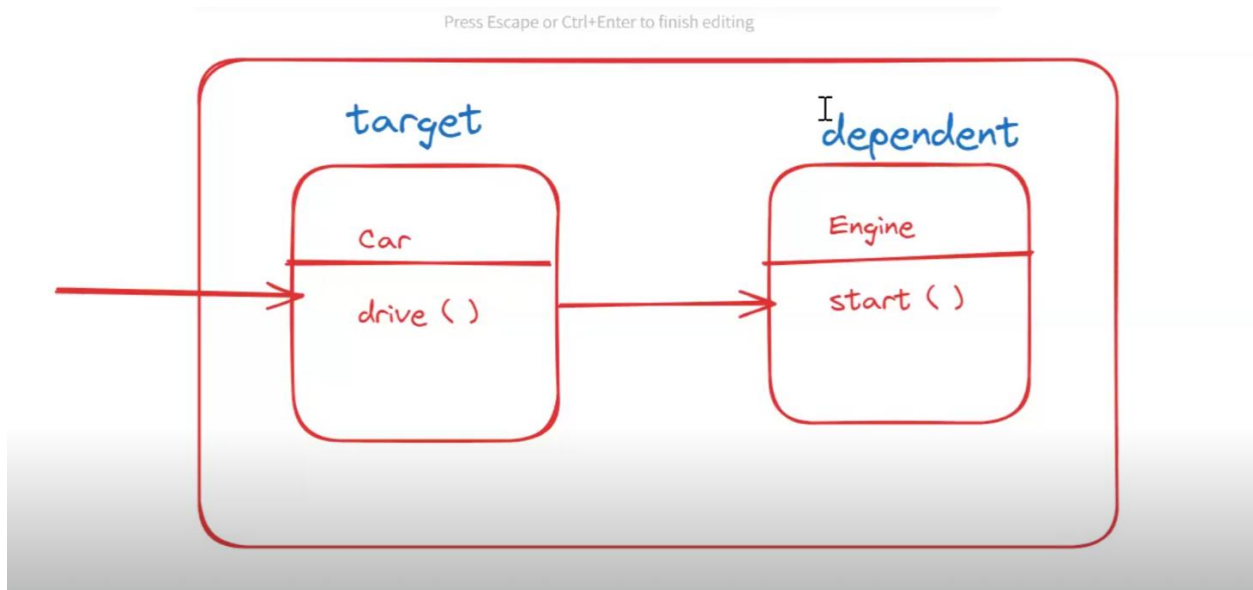
        Engine eng = new Engine();

        int engstatus = eng.start();
        if (engstatus == 1) {
            System.out.println("Journey started..");
        } else {
            System.out.println("Engine problem...");
        }
    }
}

public class Test {
    public static void main(String[] args) {
        Car c=new Car();
        c.drive();
    }
}

```

- ❖ If someone modify Engine class constructor then Car class will fail...
- ❖ with IS-A and HAS-A relation our classes are becoming tightly coupled.
- ❖ If I make change one class and it's affecting other class then it is tightly coupled. And if I make a change one class and it's not affecting to another class then it is loosely coupled.



❖ To overcome problems of IS-A and HAS-A relation we can use Spring Core module concepts.

- 1) IOC Container
- 2) Dependency Injection.

2) Dependency Injection :

Create one class object and inject into another class object is called as Dependency Injection.

1) IOC : Inversion of control

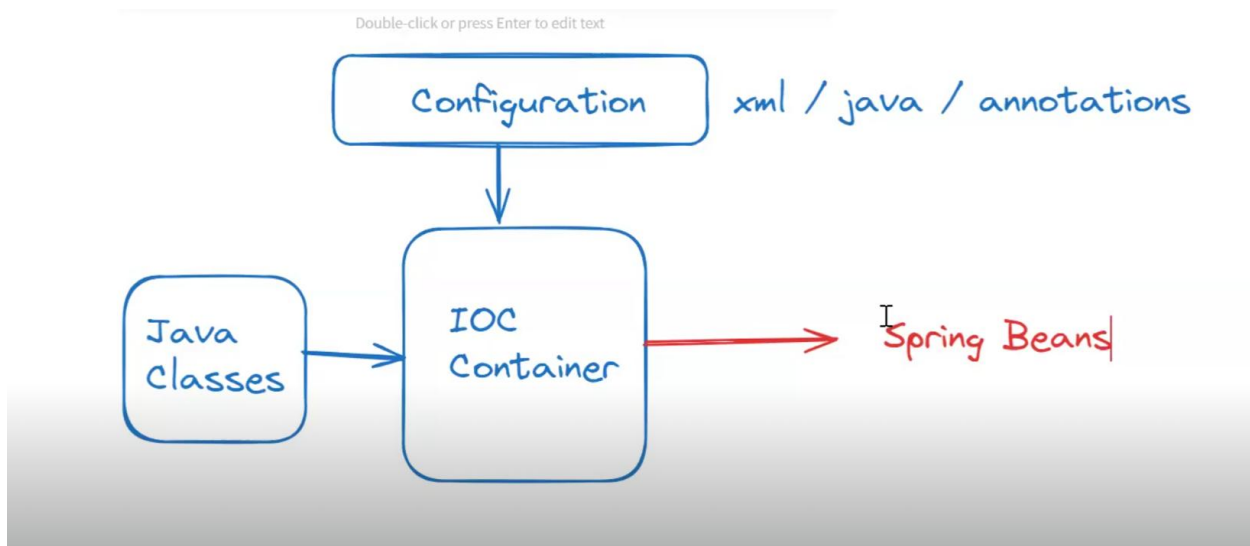
❖ IOC is a principle which is used to manage & collaborate the classes available in our application.

- ❖ IOC will perform Dependency Injection in our application.
- ❖ By using IOC and DI we can achieve Loosely coupling among the classes.
- ❖ We will give our java classes and configuration as input to IOC container.

Note: We can do configuration in 3 ways

- 1) xml (outdated -> spring boot will not support)
- 2) java based
- 3) Annotations

- ❖ IOC will take our normal java classes and it provides Spring Beans.



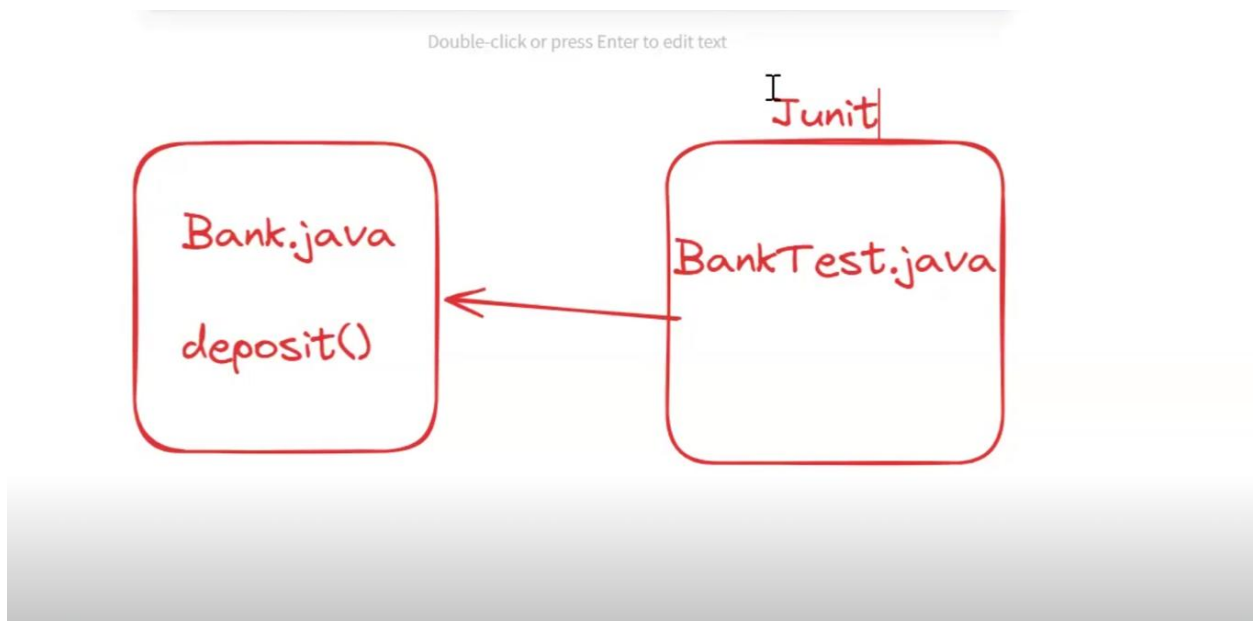
What is Spring Bean

- ❖ The java class which is managed by IOC is called as Spring bean.

First App development using Spring framework with XML

Date : 18-July-2024

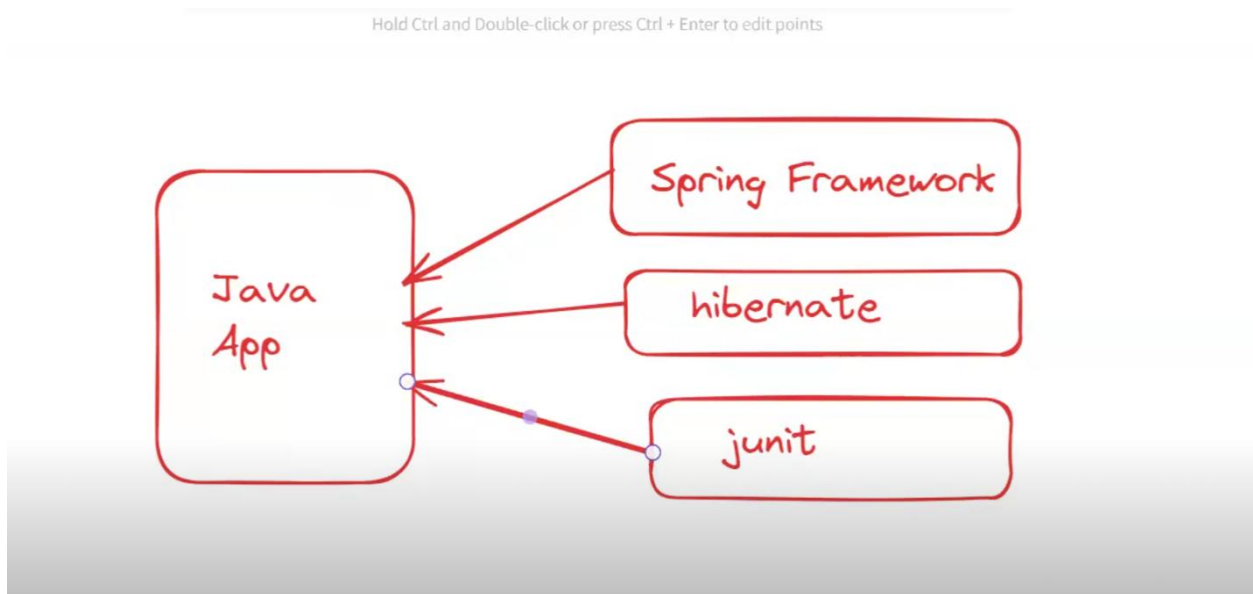
First App development using Spring framework



1) Create Maven Project

- select simple project (standalone)
- groupId : in.ashokit
- artifactId : 01-Spring-App

2) Configure Spring Framework as a dependency in pom.xml



URL : www.mvnrepository.com

<dependencies>

<dependency>

<groupId>org.springframework</groupId>

<artifactId>spring-context</artifactId>

<version>6.1.11</version>

</dependency>

</dependencies>

3) Create Java classes

4) Create Spring Bean Configuration file

Location : src/main/resources/spring-beans.xml

```

-----
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="eng" class="in.ashokit.Engine"/>

    <bean id="car" class="in.ashokit.Car"/>

</beans>
-----

```

5) Create Test class to run the application.

```

public class Test {
    public static void main(String[] args) {
        // starting IOC by giving xml as input
        ApplicationContext ctxt = new ClassPathXmlApplicationContext("spring-
beans.xml");

        Engine engine = ctxt.getBean(Engine.class);
        engine.start();
    }
}

```

What is dependency injection? How to use setter injection in XML?

Date : 19-July-2024

❖ creating one class obj and inject into another class obj is called as dependency injection.

- 1) What is dependency injection ?
- 2) How many ways we can perform dependency injection ?
- 3) What is constructor injection ?
- 4) What is setter injection >

```
public class Car {  
  
    private Engine eng; // default value is null  
  
    public Car() {  
        System.out.println("Car - Constructor...");  
    }  
  
    public void setEng(Engine eng) {  
        System.out.println("setEng() - called..");  
        this.eng = eng;  
    }  
  
    public void drive() {  
        int status = eng.start();  
        if (status == 1) {
```

```

        System.out.println("journey started..");
    } else {
        System.out.println("engine problem..");
    }
}
}

```

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           http://www.springframework.org/schema/beans/spring-beans.xsd">
    <bean id="e" class="in.ashokit.Engine" />
    <bean id="c" class="in.ashokit.Car">
        <property name="eng" ref="e" />
    </bean>
</beans>

```

```

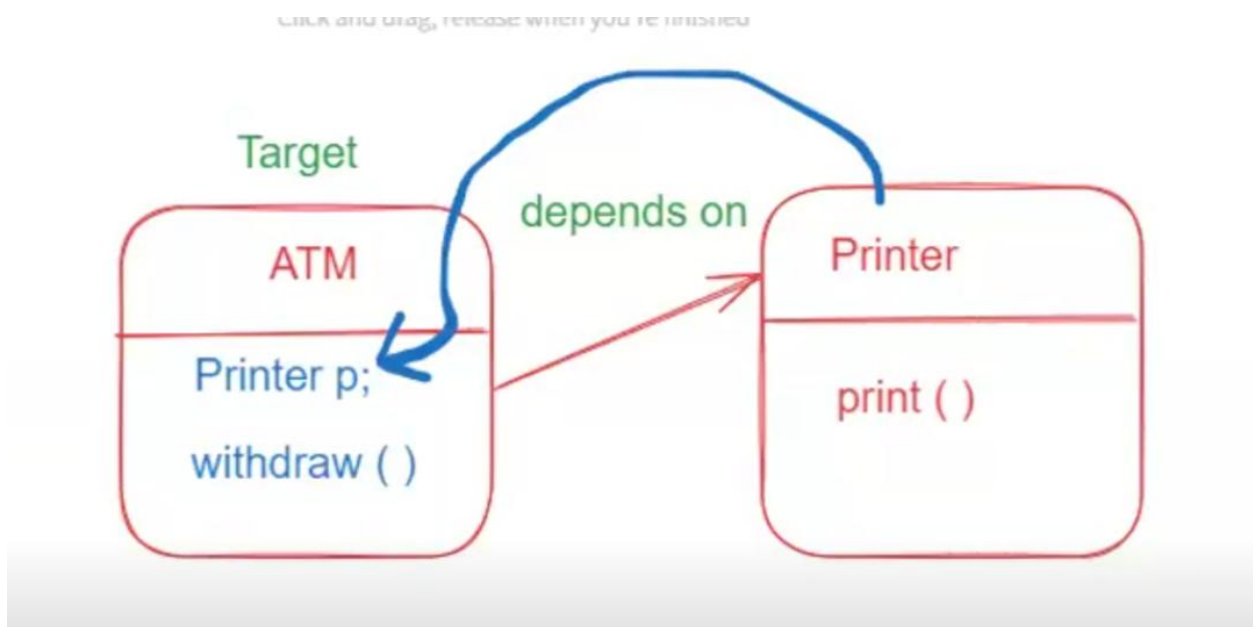
public class Test {
    public static void main(String[] args) {
        ApplicationContext ctx = new ClassPathXmlApplicationContext("spring-
beans.xml");
        Car c = ctx.getBean(Car.class);
    }
}

```

```
c.drive();  
}  
}
```

How to use constructor injection in XML?

Date : 22-July-2024



Setter injection :

- ❖ inject dependent bean obj into target bean obj using target class setter method.

ex: <property name="" ref=""/>

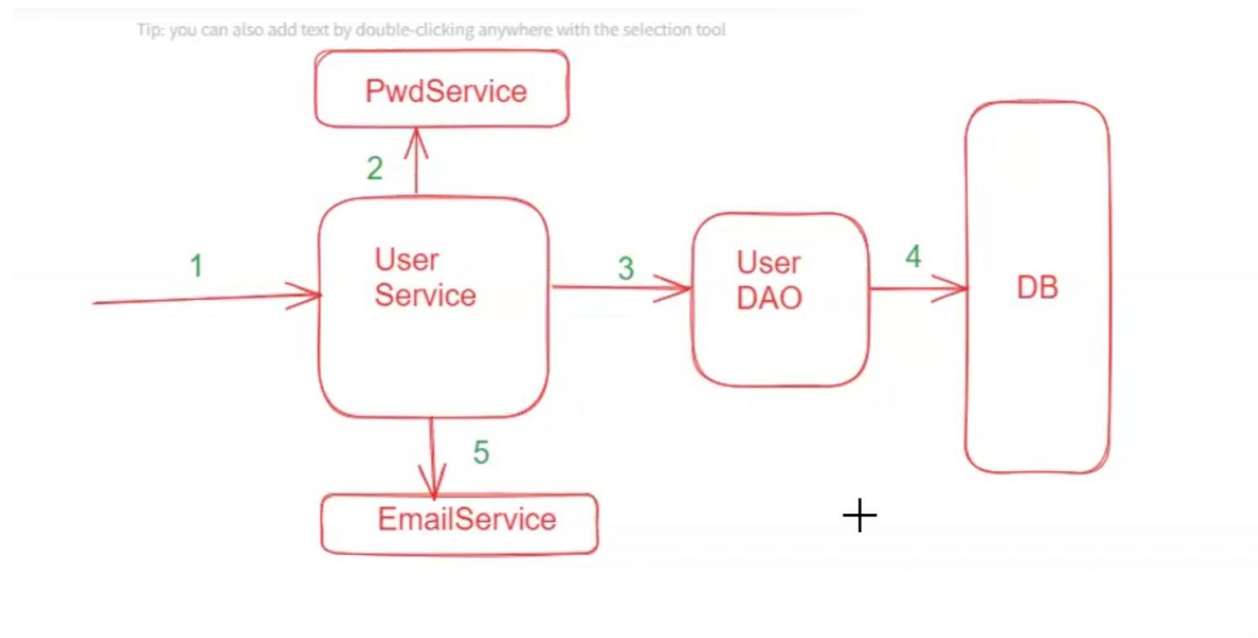
Constructor injection :

- ❖ Inject dependent bean obj into target bean obj using target class parameterized constructor.

ex: <constructor-arg name="" ref=""/>

Q) If perform both CI and SI then what will happen ?

- ❖ CI will happen at the time of obj creation
 - ❖ SI will happen after object creation
 - ❖ Hence SI will override CI value
-



```

public class Printer {
    public Printer() {
        System.out.println("Printer - Constructor...");
    }

    public void print() {
        System.out.println("Receipt printing....");
    }
}

```

```

public class ATM {
    private Printer printer;// null
    public ATM() {
        System.out.println("ATM : 0-param Constructor");
    }
}

```

```

public ATM(Printer printer) {
    System.out.println("ATM : 1-param Constructor");
    this.printer = printer;
}

public void setPrinter(Printer printer) {
    System.out.println("setter - method called..F");
    this.printer = printer;
}

public void withdraw(int amt) {
    System.out.println("amt deducted...");
    printer.print();
}

}

-----
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="
        http://www.springframework.org/schema/beans
        http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="p" class="in.ashokit.beans.Printer" />

```



```

<bean id="atm" class="in.ashokit.beans.ATM">
    <property name="printer" ref="p" />
    <constructor-arg name="printer" ref="p" />
</bean>

```

```

</beans>

```

```

public class Test {
    public static void main(String[] args) {
        ApplicationContext ctxt =
            new ClassPathXmlApplicationContext("beans.xml");

        ATM bean = ctxt.getBean(ATM.class);
        bean.withdraw(1000);
    }
}

```

Date : 23-July-2024

❖ Please watch below videos before coming to next session

Debugging : <https://youtu.be/2WxsClYhreE?si=E7X-0Qk4So4lb65K>

Eclipse Shortcuts : <https://youtu.be/TvYMey5SYa8?si=YV0mo9XH1ley6j0>

Self Introduction : <https://youtu.be/xvSO8clOmIE?si=2Q6svRaolvEY7yAU>

Roles & responsibilities : <https://youtu.be/72tbMvbXAq4?si=DUWDHLkt7cTSwj4r>

Strings Coding Challenges: <https://youtu.be/RMRkK3rF1OU?si=EKkyDz2j4SE7zO2R>

Date : 24-July-2024

- 1) Programming Languages Vs Technologies Vs Framework
- 2) What is Framework
- 3) Struts Vs Hibernate Vs Spring
- 4) Spring framework introduction
- 5) Spring Architecture
- 6) Spring Modules Overview
- 7) Spring Core Module
- 8) IOC & DI
- 9) Setter Injection (SI)
- 10) Constructor Injection (CI)